

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report
sns.set_theme(style="whitegrid")
```

```
# To upload manually:
from google.colab import files
uploaded = files.upload()

# Load into a DataFrame
df = pd.read_csv('diabetic_data.csv')
```

Choose Files diabetic_data.csv
diabetic_data.csv(text/csv) - 19159383 bytes, last modified: 5/8/2026 - 100% done
Saving diabetic_data.csv to diabetic_data.csv

```
df.head()
```

	encounter_id	patient_nbr	race	gender	age	weight	admission_type_id	discharge_disposition_id	admission_sou
0	2278392	8222157	Caucasian	Female	[0-10)	?	6		25
1	149190	55629189	Caucasian	Female	[10-20)	?	1		1
2	64410	86047875	AfricanAmerican	Female	[20-30)	?	1		1
3	500364	82442376	Caucasian	Male	[30-40)	?	1		1
4	16680	42519267	Caucasian	Male	[40-50)	?	1		1

5 rows × 50 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 101766 entries, 0 to 101765
Data columns (total 50 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   encounter_id                          101766 non-null int64
1   patient_nbr                           101766 non-null int64
2   race                                  101766 non-null object
3   gender                                101766 non-null object
4   age                                    101766 non-null object
5   weight                                101766 non-null object
6   admission_type_id                     101766 non-null int64
7   discharge_disposition_id              101766 non-null int64
8   admission_source_id                   101766 non-null int64
9   time_in_hospital                      101766 non-null int64
10  payer_code                             101766 non-null object
11  medical_specialty                      101766 non-null object
12  num_lab_procedures                     101766 non-null int64
13  num_procedures                         101766 non-null int64
14  num_medications                        101766 non-null int64
15  number_outpatient                      101766 non-null int64
16  number_emergency                       101766 non-null int64
17  number_inpatient                       101766 non-null int64
18  diag_1                                 101766 non-null object
19  diag_2                                 101766 non-null object
20  diag_3                                 101766 non-null object
21  number_diagnoses                       101766 non-null int64
22  max_glu_serum                          5346 non-null  object
23  A1Cresult                              17018 non-null object
24  metformin                              101766 non-null object
25  repaglinide                            101766 non-null object
26  nateglinide                            101766 non-null object
27  chlorpropamide                         101766 non-null object
28  glimepiride                            101766 non-null object
29  acetohexamide                          101766 non-null object
30  glipizide                              101766 non-null object
31  glyburide                              101766 non-null object
32  tolbutamide                            101766 non-null object
33  pioglitazone                           101766 non-null object
34  rosiglitazone                          101766 non-null object
35  acarbose                               101766 non-null object
```

```

36 miglitol          101766 non-null object
37 troglitazone      101766 non-null object
38 tolazamide        101766 non-null object
39 examide           101766 non-null object
40 citoglipton       101766 non-null object
41 insulin           101766 non-null object
42 glyburide-metformin 101766 non-null object
43 glipizide-metformin 101766 non-null object
44 glimepiride-pioglitazone 101766 non-null object
45 metformin-rosiglitazone 101766 non-null object
46 metformin-pioglitazone 101766 non-null object
47 change            101766 non-null object
48 diabetesMed       101766 non-null object
49 readmitted        101766 non-null object
dtypes: int64(13), object(37)
memory usage: 38.8+ MB

```

```
df.describe()
```

	encounter_id	patient_nbr	admission_type_id	discharge_disposition_id	admission_source_id	time_in_hospital	num_lab
count	1.017660e+05	1.017660e+05	101766.000000	101766.000000	101766.000000	101766.000000	
mean	1.652016e+08	5.433040e+07	2.024006	3.715642	5.754437	4.395987	
std	1.026403e+08	3.869636e+07	1.445403	5.280166	4.064081	2.985108	
min	1.252200e+04	1.350000e+02	1.000000	1.000000	1.000000	1.000000	
25%	8.496119e+07	2.341322e+07	1.000000	1.000000	1.000000	2.000000	
50%	1.523890e+08	4.550514e+07	1.000000	1.000000	7.000000	4.000000	
75%	2.302709e+08	8.754595e+07	3.000000	4.000000	7.000000	6.000000	
max	4.438672e+08	1.895026e+08	8.000000	28.000000	25.000000	14.000000	

```

## Data Exploration and Cleaning
##Handling missing value##

```

```
df.replace('?', np.nan, inplace=True)
```

```
print(df.isnull().sum())
```

```

encounter_id          0
patient_nbr           0
race                  2273
gender                0
age                   0
weight               98569
admission_type_id     0
discharge_disposition_id 0
admission_source_id   0
time_in_hospital      0
payer_code            40256
medical_specialty     49949
num_lab_procedures    0
num_procedures        0
num_medications       0
number_outpatient     0
number_emergency      0
number_inpatient      0
diag_1                 21
diag_2                 358
diag_3                1423
number_diagnoses      0
max_glu_serum         96420
A1Cresult             84748
metformin             0
repaglinide           0
nateglinide           0
chlorpropamide        0
glimepiride           0
acetohexamide         0
glipizide             0
glyburide             0
tolbutamide           0
pioglitazone          0
rosiglitazone         0
acarbose              0
miglitol              0
troglitazone          0
tolazamide            0
examide               0
citoglipton           0
insulin               0

```

```
glyburide-metformin      0
glipizide-metformin      0
glimepiride-pioglitazone  0
metformin-rosiglitazone  0
metformin-pioglitazone   0
change                   0
diabetesMed               0
readmitted                0
dtype: int64
```

```
df.drop(columns=['medical_specialty', 'max_glu_serum', 'A1Cresult'], inplace=True)
```

```
df.dropna(subset=['diag_1'], inplace=True)
```

```
df['diag_2'] = df['diag_2'].fillna('None')
df['diag_3'] = df['diag_3'].fillna('None')
```

```
df['race'] = df['race'].fillna('Other')
```

```
df.drop(columns=['weight', 'payer_code'], inplace=True)
```

```
if 'patient_nbr' in df.columns:
    df.drop_duplicates(subset='patient_nbr', keep='first', inplace=True)
```

```
print("Remaining Nulls:")
print(df.isnull().sum())
print(f"\nFinal Dataset Shape: {df.shape}")
```

```
Remaining Nulls:
encounter_id      0
patient_nbr       0
race               0
gender             0
age               0
admission_type_id  0
discharge_disposition_id  0
admission_source_id  0
time_in_hospital  0
num_lab_procedures  0
num_procedures    0
num_medications   0
number_outpatient  0
number_emergency   0
number_inpatient   0
diag_1            0
diag_2            0
diag_3            0
number_diagnoses   0
metformin         0
repaglinide       0
nateglinide       0
chlorpropamide    0
glimepiride       0
acetohexamide     0
glipizide         0
glyburide         0
tolbutamide       0
pioglitazone      0
rosiglitazone     0
acarbose         0
miglitol          0
troglitazone      0
tolazamide        0
examide           0
citoglipton       0
insulin           0
glyburide-metformin  0
glipizide-metformin  0
glimepiride-pioglitazone  0
metformin-rosiglitazone  0
metformin-pioglitazone  0
change           0
diabetesMed       0
readmitted        0
dtype: int64
```

```
Final Dataset Shape: (71509, 45)
```

```
# Create a binary column: 1 for readmitted <30 days, 0 otherwise
df['readmission_risk'] = df['readmitted'].apply(lambda x: 1 if x == '<30' else 0)
```

```
# Verify the split
print(df['readmission_risk'].value_counts(normalize=True) * 100)
```

```
readmission_risk
0    91.202506
1     8.797494
Name: proportion, dtype: float64
```

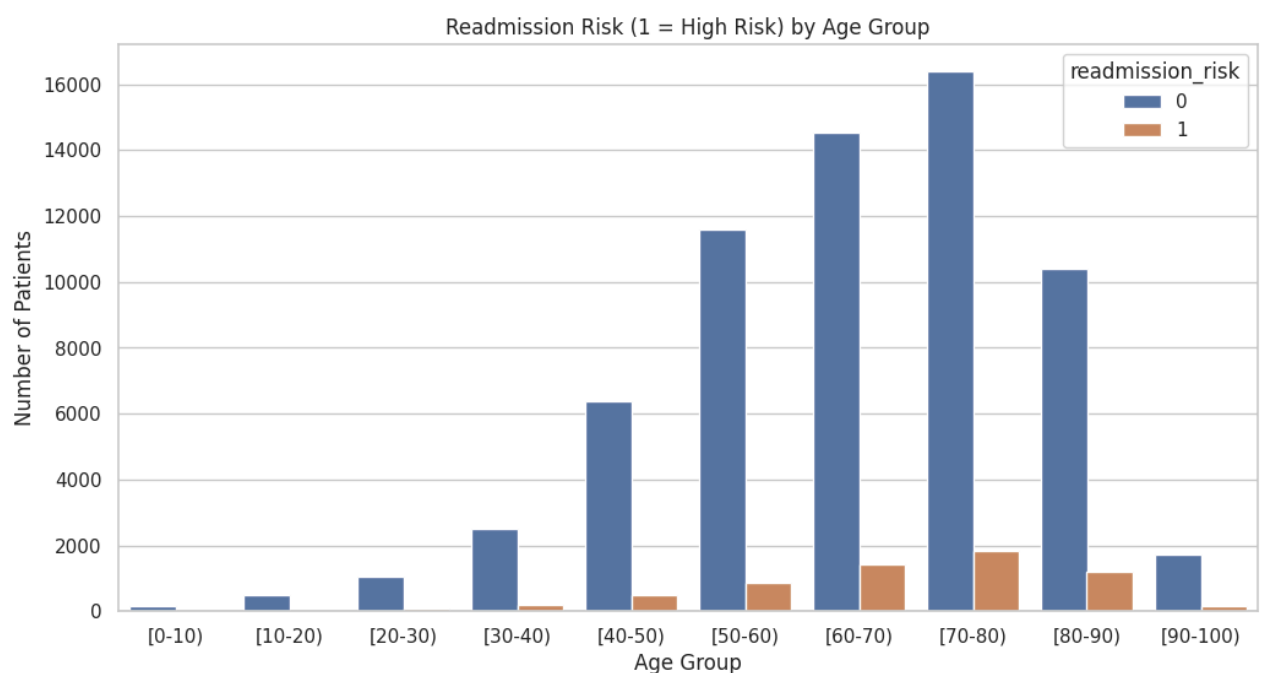
```
print(df.shape)
```

```
(71509, 46)
```

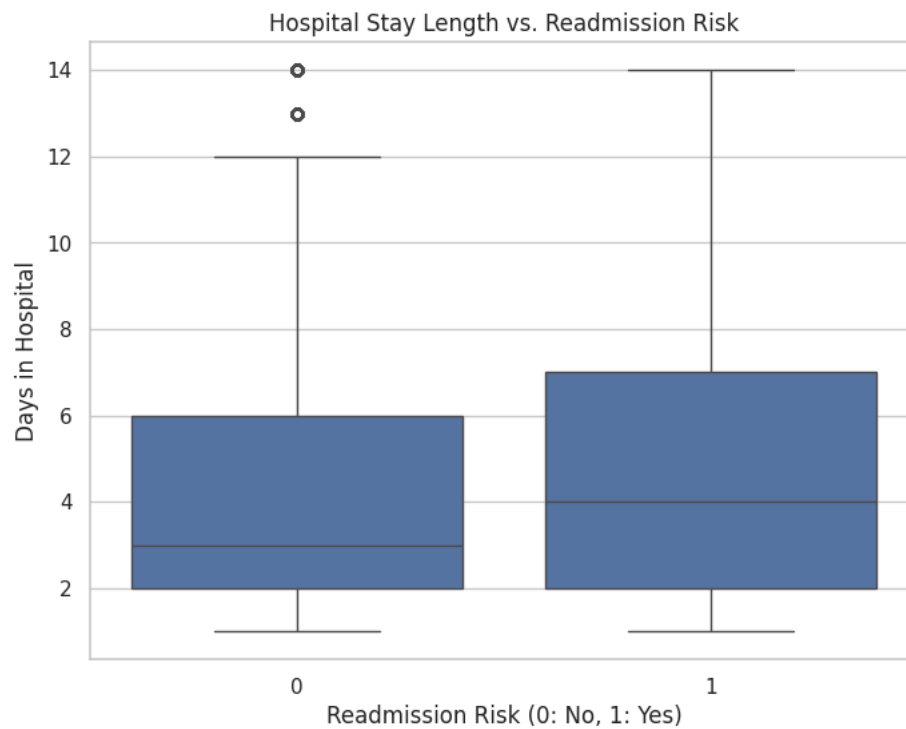
```
## EDA
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
plt.figure(figsize=(12,6))
# This shows how many people in each age group were high risk (1) vs low risk (0)
sns.countplot(data=df, x='age', hue='readmission_risk')
plt.title('Readmission Risk (1 = High Risk) by Age Group')
plt.xlabel('Age Group')
plt.ylabel('Number of Patients')
plt.show()
```

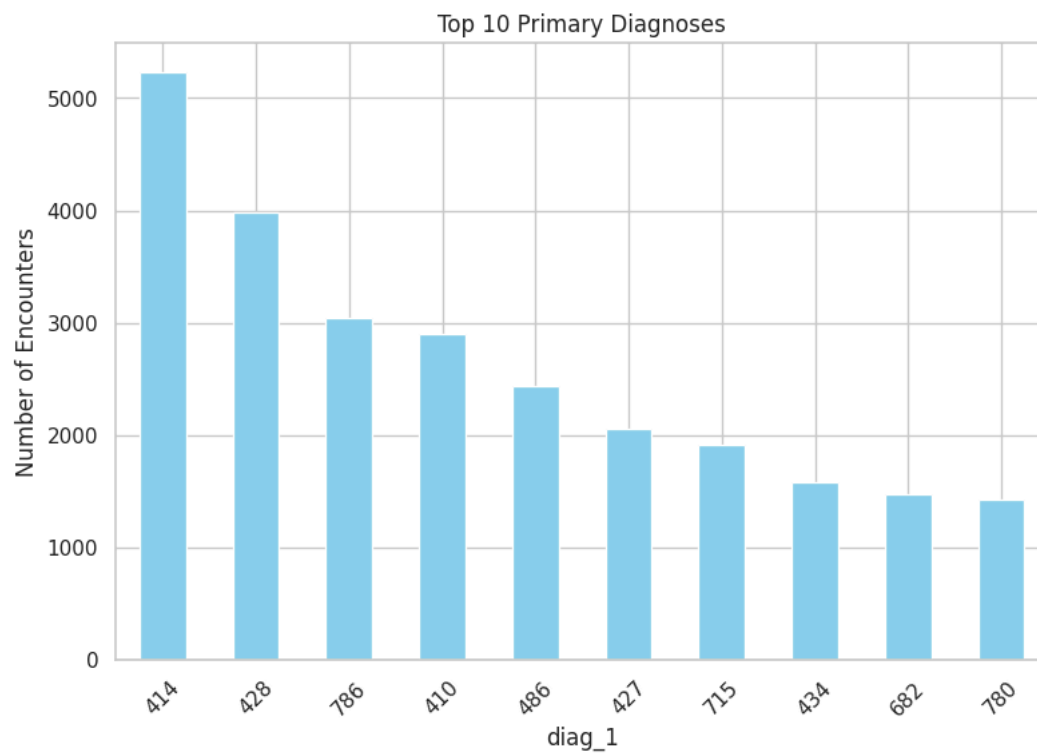


```
plt.figure(figsize=(8,6))
# Boxplot shows the distribution of stay length for both groups
sns.boxplot(data=df, x='readmission_risk', y='time_in_hospital')
plt.title('Hospital Stay Length vs. Readmission Risk')
plt.xlabel('Readmission Risk (0: No, 1: Yes)')
plt.ylabel('Days in Hospital')
plt.show()
```



```
# Show the top 10 primary diagnoses
top_diagnoses = df['diag_1'].value_counts().head(10)

plt.figure(figsize=(9,6))
top_diagnoses.plot(kind='bar', color='skyblue')
plt.title('Top 10 Primary Diagnoses')
plt.ylabel('Number of Encounters')
plt.xticks(rotation=45)
plt.show()
```



```
## SQL
```

```
import sqlite3

# 1. Connect to a temporary in-memory database
conn = sqlite3.connect(':memory:')

# 2. Push your cleaned DataFrame into the SQL database
```

```
df.to_sql('diabetes_data', conn, index=False)

# 3. SQL Query: Readmission rate by Race (using GROUP BY + HAVING as requested)
query = """
SELECT
    race,
    COUNT(*) as total_patients,
    SUM(readmission_risk) as high_risk_count,
    ROUND(CAST(SUM(readmission_risk) AS FLOAT) / COUNT(*) * 100, 2) as readmission_rate_pct
FROM diabetes_data
GROUP BY race
HAVING total_patients > 100
ORDER BY readmission_rate_pct DESC
"""

# Execute and view results
res = pd.read_sql_query(query, conn)
print(res)
```

	race	total_patients	high_risk_count	readmission_rate_pct
0	Caucasian	53489	4816	9.00
1	AfricanAmerican	12884	1093	8.48
2	Asian	496	41	8.27
3	Hispanic	1515	120	7.92
4	Other	3125	221	7.07

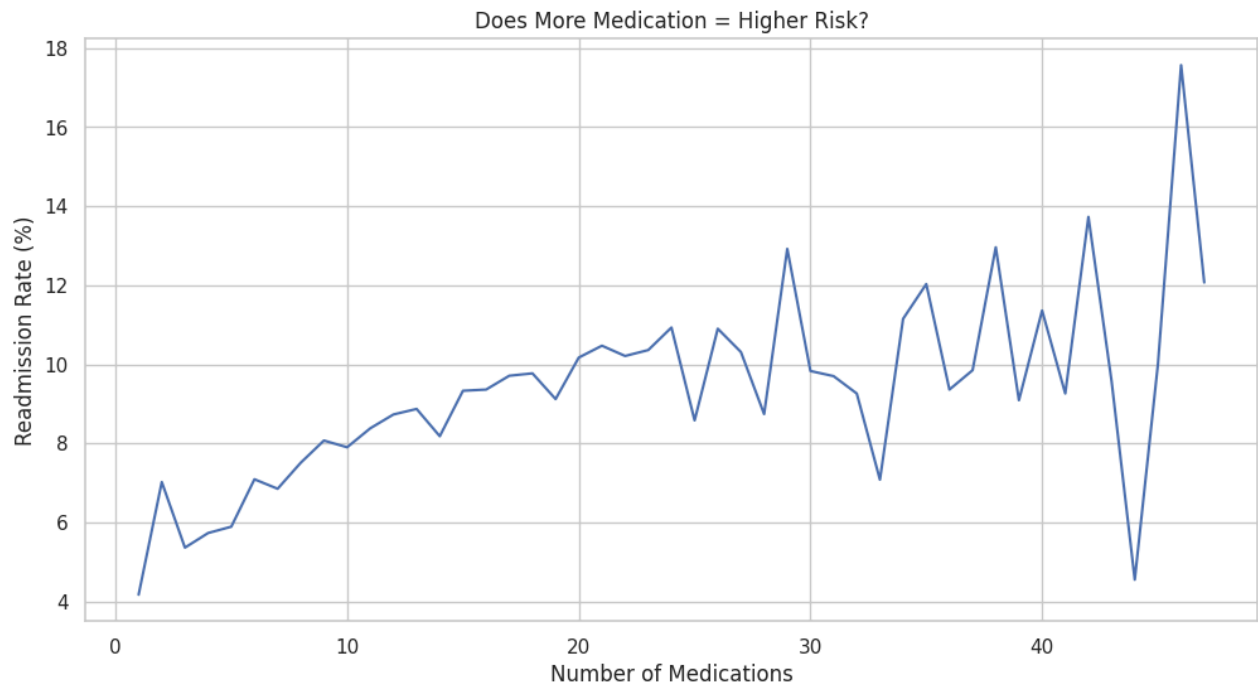
```
# checking readmission by the number of medications
query_meds = """
SELECT
    num_medications,
    COUNT(*) as total_patients,
    ROUND(AVG(readmission_risk) * 100, 2) as readmission_rate_pct
FROM diabetes_data
GROUP BY num_medications
HAVING total_patients > 50
ORDER BY num_medications ASC
"""

res_meds = pd.read_sql_query(query_meds, conn)
print(res_meds)
```

	num_medications	total_patients	readmission_rate_pct
0	1	216	4.17
1	2	399	7.02
2	3	727	5.36
3	4	1169	5.73
4	5	1631	5.89
5	6	2129	7.09
6	7	2701	6.85
7	8	3304	7.51
8	9	3668	8.07
9	10	3913	7.90
10	11	4080	8.38
11	12	4320	8.73
12	13	4330	8.87
13	14	3913	8.18
14	15	3954	9.33
15	16	3706	9.36
16	17	3326	9.71
17	18	3018	9.77
18	19	2664	9.12
19	20	2410	10.17
20	21	2102	10.47
21	22	1881	10.21
22	23	1603	10.36
23	24	1354	10.93
24	25	1247	8.58
25	26	1064	10.90
26	27	931	10.31
27	28	824	8.74
28	29	650	12.92
29	30	539	9.83
30	31	464	9.70
31	32	432	9.26
32	33	353	7.08
33	34	296	11.15
34	35	266	12.03
35	36	203	9.36
36	37	203	9.85
37	38	162	12.96
38	39	154	9.09
39	40	132	11.36
40	41	108	9.26
41	42	102	13.73
42	43	94	9.57
43	44	66	4.55

44	45	70	10.00
45	46	74	17.57
46	47	58	12.07

```
plt.figure(figsize=(12,6))
sns.lineplot(data=res_meds, x='num_medications', y='readmission_rate_pct')
plt.title('Does More Medication = Higher Risk?')
plt.xlabel('Number of Medications')
plt.ylabel('Readmission Rate (%)')
plt.show()
```

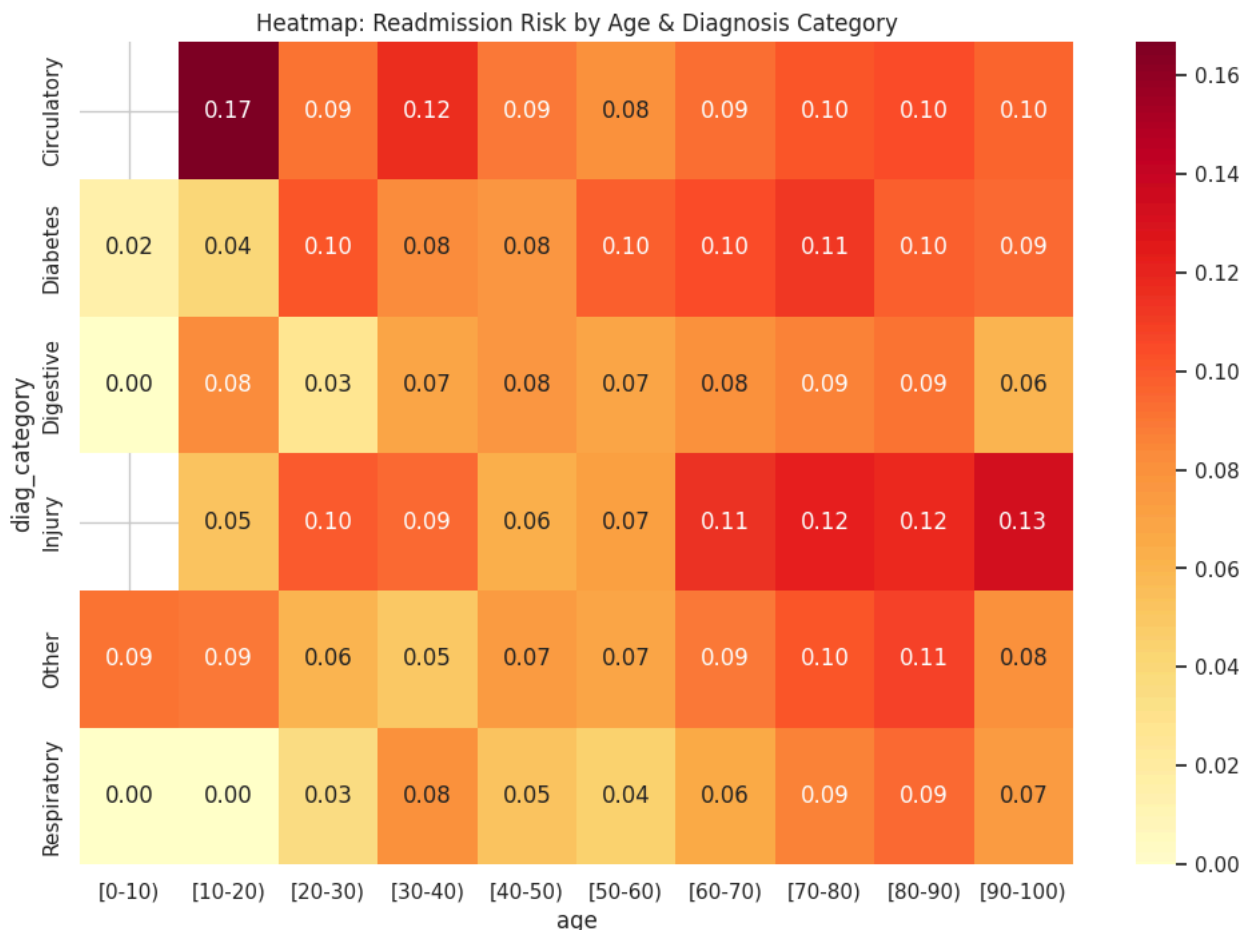


```
# 1. Create a helper function to categorize diagnoses
def categorize_diag(code):
    try:
        code = float(code)
        if 390 <= code <= 459 or code == 785: return 'Circulatory'
        if 460 <= code <= 519 or code == 786: return 'Respiratory'
        if 520 <= code <= 579 or code == 787: return 'Digestive'
        if 250 <= code < 251: return 'Diabetes'
        if 800 <= code <= 999: return 'Injury'
        return 'Other'
    except:
        return 'Other'

# 2. Apply categorization
df['diag_category'] = df['diag_1'].apply(categorize_diag)

# 3. Create a Heatmap (Diagnoses vs. Age vs. Readmission)
pivot_table = df.pivot_table(index='diag_category', columns='age', values='readmission_risk', aggfunc='mean')

plt.figure(figsize=(12, 8))
sns.heatmap(pivot_table, annot=True, cmap='YlOrRd', fmt='.2f')
plt.title('Heatmap: Readmission Risk by Age & Diagnosis Category')
plt.show()
```



```
# downloading the cleaned dataset--
```

```
df.to_csv('cleaned_hospital_data.csv', index=False)
from google.colab import files
files.download('cleaned_hospital_data.csv')
```

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
from sklearn.preprocessing import LabelEncoder

# 1. Select the most relevant features based on your EDA
features = ['age', 'race', 'diag_category', 'num_medications', 'time_in_hospital', 'num_lab_procedures']
X = df[features].copy()
y = df['readmission_risk']

# 2. Encode categorical text into numbers
le = LabelEncoder()
for col in ['age', 'race', 'diag_category']:
    X[col] = le.fit_transform(X[col].astype(str))

# 3. Split the data: 80% for training, 20% for testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
# Initialize the model
model = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42)

# Train the model
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)
```

```
print(f"Overall Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("\nDetailed Performance Report:")
print(classification_report(y_test, y_pred))
```

```
# Visualize which features mattered most
importances = pd.Series(model.feature_importances_.index=features).sort_values(ascending=False)
```

```
plt.figure(figsize=(10,6))
sns.barplot(x=importances, y=importances.index)
plt.title('Which factors predict readmission best?')
plt.show()
```

Overall Accuracy: 0.91

Detailed Performance Report:

	precision	recall	f1-score	support
0	0.91	1.00	0.95	13044
1	0.00	0.00	0.00	1258
accuracy			0.91	14302
macro avg	0.46	0.50	0.48	14302
weighted avg	0.83	0.91	0.87	14302

